

Fiche de Révision : C & Mémoire

Résumé du cours magistral d'introduction au langage C

1. ORGANISATION & ÉVALUATION

Le cours est pensé de manière interactive. L'évaluation s'écarte de l'examen terminal classique pour privilégier :

- Des **évaluations individuelles à l'oral**.
- Des **petits contrôles** réguliers en début de CM. La moins bonne note est retirée (principe du joker), et la moyenne est faite sur les autres.
- Un projet imposant des contraintes techniques (ex. utilisation de listes chaînées).

2. CARACTÉRISTIQUES DU LANGAGE C

Le C est un langage de **bas niveau**, ancien (années 70/80, avec plusieurs révisions jusqu'en 2018), qui donne au programmeur un contrôle total sur l'allocation et la gestion de la mémoire, contrairement aux langages interprétés comme Python.

3. GESTION DE LA MÉMOIRE

L'allocation de la mémoire se fait au moment de **l'exécution** (et non à la compilation). Le programme divisera la mémoire en plusieurs espaces :

A. La Pile (Stack)

- **Contenu** : Variables locales, paramètres de fonctions, valeurs de retour, et **adresses de retour**.
- **Fonctionnement** : Fonctionne selon le principe d'empilage/dépilage (LIFO). Permet de sauvegarder l'état avant d'entrer dans une fonction et d'y revenir à la sortie (crucial pour la récursivité).
- **Durée de vie** : Gérée automatiquement et rapidement. Les variables locales sont détruites à la fin de la fonction.

B. Le Tas (Heap)

- **Contenu** : Mémoire allouée dynamiquement (ex. avec `malloc()`).
- **Usage** : Idéal pour les grandes structures de données dont la taille n'est pas connue à l'avance (ex. listes chaînées, gros tableaux).
- **Contrôle** : Doit être gérée (et libérée) explicitement par le développeur.

C. Le Segment de Code

- **Contenu** : Contient les instructions du programme (le code transformé en langage machine), incluant le code de toutes les fonctions.

4. COMPILEATION VS INTERPRÉTATION

Compilation (C) : Traduit le code source complet en un langage machine exécutable par le processeur. On peut compiler par morceaux (fichier par fichier) générant des fichiers objets (.o).

Interprétation (Python, etc.) : Le code est traduit et exécuté instruction par instruction à la volée. Plus lent et nécessite l'interpréteur à chaque lancement.

5. COMPILEATION ET ÉDITION DE LIENS (LINKAGE)

La création d'un exécutable en C se fait en deux étapes majeures :

Étape	Description
1. Compilation (gcc -c)	Transforme les fichiers source (.c) en fichiers objets (.o) indépendants.
2. Édition de liens / Linkage	Rassemble et relie ("link") tous les fichiers .o ainsi que les bibliothèques pour créer l'exécutable final. Sans cette étape, le code n'est pas exécutable.

6. BIBLIOTHÈQUES (LIBRARIES)

Bibliothèque Statique (.a)	Bibliothèque Dynamique (.so)
Le code de la bibliothèque est copié physiquement et inclus dans l'exécutable lors de l'édition de liens.	Le code de la bibliothèque n'est pas inclus ; l'exécutable y fait référence et la charge lors de l'exécution.
Avantage : Le programme est autonome. Pas de risque de dépendance manquante.	Avantage : Exécutable très léger. La bibliothèque peut être partagée en RAM par plusieurs programmes.
Inconvénient : Taille de l'exécutable beaucoup plus grande. Si 3 programmes l'utilisent, le code est dupliqué 3 fois sur le disque.	Inconvénient : Risque d'erreurs d'exécution si le fichier .so est introuvable ou mal placé.

7. STRUCTURE DES FICHIERS (.C ET .H)

- **Fichiers .c** : Contiennent le code source réel (l'implémentation des fonctions).

- **Fichiers Headers .h (En-têtes)** : Contiennent les prototypes (signatures) des fonctions et les définitions de types. Ils servent à "rassurer" le compilateur en lui indiquant qu'une fonction existe ailleurs avant l'étape de l'édition de liens.
- L'instruction `#include` permet d'importer le contenu de ces fichiers avant même le début de la compilation.